

Hvordan refleksjon bidrar til læring i smidig utvikling¹

I denne essayet skal jeg ta for meg hvordan refleksjon bidrar til læring i smidig utvikling. Jeg skal fokusere på tre områder, retrospektiver, daglig scrum og programmeringspraksiser. Først skal jeg ta opp relevant teori rundt disse deretter skal jeg ta opp noen erfaringer vi hadde i teamet, og til slutt skal jeg drøfte problemstillingen i lys av både teori og erfaringer.

Teori om refleksjon og læring i smidig utvikling

Det første som er naturlig å ta opp er retrospektiver. Kniberg mener at dette er en av de viktigste aspektene ved scrum, og anbefaler alle teams å gjennomføre dem “Why we insist that all teams do retrospectives” (Kniberg, 2015, s. 84). Hovedmålet med retrospektivmøtene er at teamet skal lære av sprinten, og dermed utvikle seg i en positiv retning. Kniberg formulerer dette som “What can we do better next sprint?” (Kniberg, 2015, s. 85). Dette oppnås med å reflektere over erfaringene teamet hadde fra sprinten, og eventuelle problemer som oppstod. En måte å gjennomføre et retrospektivmøte på er å fokusere på tre ting: Det som var bra, det som kunne vært bedre, og til slutt konkrete tiltak for neste sprint (Kniberg, 2015, s. 86). Dette var modellen som vi valgte å bruke for våre retrospektiver. Et viktig aspekt ved retrospektivene er at det må være en handlingsplan for neste sprint. Altså at det velges ut noen konkrete tiltak som teamet skal prøve å gjennomføre neste sprint, og helst at noen i teamet er ansvarlig for at disse tiltakene går gjennom. Hvis dette ikke gjøres kan det medføre at det kun blir diskusjon, og ikke noe handling og dermed ingen forbedring.

Daily standup, eller en annen form for daglig scrum, spiller også en viktig rolle for refleksjonen i teamet. For å få en god retrospektiv må teamet være bevisst på hvordan sprinten gikk og daily standups hjelper veldig med dette. De opplyser teamet både om hva som blir gjort, og også hvilke problemer som oppstår underveis. Dette gjør det lettere å oppdage mønstre, som f.eks at det er dårlig testdekning. Dette støttes av Stray (2020, s. 70–72), som viser at daily stand-up kan gi bedre informasjonsdeling, rask problemidentifisering og bedre koordinering i teamet.

Hlazunova (2025, s.12) viser også at sprintbasert arbeid og løpende tilbakemeldinger kan styrke engasjement, samarbeid og selvstendig læring. Dette støtter tanken om at refleksjon i smidig utvikling ikke bare skjer i retrospektiver, men også kontinuerlig gjennom sprinten. I denne konteksten er det selvfølgelig også relevant å ta opp ‘The Reflective Agile Learning Model (REALM)’ (Babb, 2014, s. 51). Artikkelen framhever at refleksjon ikke bare skjer i retrospektivene, men gjennom hele sprinten, og inneholder en tabell som illustrerer dette, se figur 1. Den skiller mellom ‘Reflection-in-action’, som betyr at en reflekterer samtidig som en arbeider, og ‘Reflection-on-action’, som betyr at en reflekterer i etterkant av arbeid. Den viser blant annet at praksiser som parprogrammering og gruppeprogrammering, også kalt mobprogrammering, gir ‘Reflection-in-action’ ved at en får et annet perspektiv for hvordan en kan tenke og programmere.

¹ Ingen bruk av kunstig intelligens

TABLE 1

Learning and reflection opportunities in the Reflective Agile Learning Model (REALM).

Agile practices	Reflection-in-action
Release planning	Learning about the project, domain, and product
Collective estimation and iteration planning	Learning about each other's skills and areas of expertise, and the team's collective capacity to delivery (velocity)
Group programming	Learning new skills and expertise from each other
Pair-in-need	Novices learning about the team and project as they pair with experts; pairs learning a new technology or skill as they work on a challenging task
Project wiki	Capturing project-based learning about the domain and individual reflections (blogs)
Demo and acceptance testing	Learning from acceptance test failures that fuel refinements and guide product vision
Agile practices	Reflection-on-action
Daily standup	Reflection on action performed the previous day, planning for the day's actions, and highlighting impediments
Learning spike	Adapted practice that embodies reflection on action as some members devote exclusive time to new learning
Retrospective	Standard agile practice devoted exclusively to reflection on actions performed in previous iteration

Figur 1: Refleksjonsmuligheter i REALM (Babb, 2014, s. 53)

Samtidig viser forskning at smidige arbeidsformer kan hemme læring i store organisasjoner. Annosi (2020, s. 78-79) tyder på at smidig arbeid i større organisasjoner kan svekke læring dersom teamene får for lite tid til refleksjon, kunnskapsdeling og systematisk forbedringsarbeid.

Erfaringer i teamet

I teamet vårt hadde vi totalt to retrospektivmøter, en for hver gjennomgang. Vi opplevde at disse var nyttig og at de hjalp oss å utvikle oss som team. F.eks var det en rød tråd på første retrospektiv at vi ville ha bedre innsikt i hva de andre på teamet arbeidet med. Vi utarbeidet deretter noen tiltak for å adressere dette, blant annet å bruke mer tid på daily standup. Alle tiltakene ble ikke fulgt fra punkt til prikke, vi ble f.eks ikke noe særlig bedre å assigne oss selv på issues, men det viktige er at alle var enige i påfølgende retrospektiv at vi hadde bedre innsikt i hva de andre arbeidet med.

Vi tok også i bruk en slags daglig scrum med daily standup når vi møttes fysisk, som vanligvis var to ganger i uken. Denne var relativt uformell siden vi satt fysisk sammen i 2-4 timer, og kunne da spørre dem andre hvis vi hadde noen spørsmål. Derfor brukte vi nok litt for lite tid på den i første sprint, men vi fikk adressert dette i første retrospektiv. Deretter ga det stor gevinst å være bedre orientert på hva som var utfordrende for de andre. Spesielt med hvordan de andre erfarte koden som en har skrevet, som f.eks at den hadde dårlig dokumentasjon. Slike småting er nok for små og spesifikke for å tas opp på retrospektiver, og daglig scrum gir dermed en mulighet for kontinuerlig tilbakemelding. Men det er viktig at samtalen holdes relevant, som var et punkt på teamets andre retrospektiv, og at det ikke f.eks er snakk om hvem som er den kuleste foreleseren (riktig svar er Børge).

I tillegg til dette benyttet vi også praksisene parprogrammering og mobprogrammering. Alle i teamet var enige i at parprogrammering var en god praksis å ha. Siden de fleste i teamet ikke var godt kjent med teknologistakken som ble brukt, hadde de fleste en del kunnskapshull som parprogrammeringen kunne hjelpe med, ved at den andre personen hadde ulike kunnskapshull. Mobprogrammering derimot var en stor fiasko, og det skyldes hovedsakelig i at det ikke var storskjerm i rommet som vi booket selv om det egentlig skulle vært det, og det endte da opp med at vi sendte en laptop på rundgang. Det var desidert det minst effektive møtet vi har hatt, og selv om tekniske problemer spilte en stor rolle, så var det en følelse i teamet at det var litt mange kokker på samme kjøkken.

Teori og erfaring i lys av hverandre

Så hvordan bidrar refleksjon til læring i smidig utvikling? Refleksjon innebærer ikke bare å identifisere problemer men også å drøfte hvordan en evt skal løse dem. Og det erfaringene våre viser er at refleksjon bidrar mest til læring ved å omgjøre erfaringer til konkrete tiltak, og dermed forbedring. Teorien framhever spesielt retrospektiver som et eksempel på dette, og det samsvarer med erfaringene til teamet om retrospektiver. Selv om ikke alle tiltakene ble fulgt, og kanskje det hadde vært gunstig om flere av dem ble det, så fikk vi en god del forbedring i påfølgende sprinter som resultat av den første retrospektiven, og vi ville nok sett det samme resultatet fra den andre sprinten. Så i likhet med Kniberg mener jeg at alle team bør ta i bruk retrospektiver på en form som passer teamet.

Men refleksjon i smidig utvikling skjer ikke bare i retrospektivene, men kontinuerlig gjennom sprinten, som ved daglig scrum. Våre erfaringer tyder på at denne refleksjonen førte til at teamet lærte hva de andre opplevde som krevende, og kunne dermed endre egne vaner for å adressere dette. Daglig scrum hjelper også med at teamet er bevisst over overordnede problemer som kan tas opp på retrospektiver. Samtidig må samtalen holdes relevant og det må allokere nok tid ellers kan gevinsten bli svekket i stor grad. Det finnes også mange gode alternativer til daglig scrum, som issue boards og kanban, men det viktige i denne sammenhengen er at teamet kommuniserer godt om problemene som oppstår. Derfor mener jeg at daglig scrum ikke kanskje er for alle, men det er hvertfall gunstig at teamet kommuniserer godt om problemer, siden dette fører til både god refleksjon og læring.

REALM påstår at refleksjon også skjer i handling, og våre erfaringer med parprogrammering støtter dette. Parprogrammering tvinger begge i paret til å se ting fra et annet perspektiv, og en kan da reflektere over om den andres framgangsmåte er bedre, og deretter justere sin egen. En må også forklare og begrunne egne valg, og får da en dypere forståelse på hva en kan og ikke kan. Det er ofte slik at en tror en kan noe, men når en skal forklare, det innser en at en ikke har like god kontroll som en første trodde. Dermed er parprogrammering noe som jeg absolutt anbefaler andre team å prøve ut. Mobprogrammering derimot er jeg mer skeptisk mot, men det skal sies at teamets erfaring var i stor grad påvirket av tekniske utfordringer, så dette er nok noe vi ville gitt et nytt forsøk.

Alt i alt tyder både teorien og erfaringene våre i teamet på at å reflektere over utfordringer teamet møter på, og prøve å lære av disse gir stor gevinst. Ingen team starter perfekt, så det er derfor viktig å legge opp til hvordan man skal adressere eventuelle problemer, slik at teamet

kan utvikle seg. Dette kan f.eks være retrospektiver, daglig scrum, eller programmeringspraksis som parprogrammering og mobprogrammering.

Litteraturliste

Annosi, Maria Carmela. Foss, Nicolai. Martini, Antonella (november 2020) *When Agile Harms Learning and Innovation: (and What Can Be Done About It)*. California management review, Vol.63 (1), s.61-80. Universitetsbiblioteket

Babb, J., Hoda, R. & Nørbjerg, J. (2014). *Embedding reflection and learning into agile software development*. IEEE software. Vol. 31(4), s. 51-57

Hlazunova, Olena. Korolchuk, Valentyna. Voloshyna, Tetiana. Kravchenko, Volodymyr. Saiapina, Taisiia. Sydoruk, Yelyzaveta (januar 2026) *Experiential learning through agile learning methodologies: understanding students' experience* Higher education, skills and work-based learning, s.1-16. Universitetsbiblioteket

Kniberg, Henrik (2015) *Scrum and XP from the trenches 2.utgave*, ISBN: 978-1-392-22427-8

Stray, Viktoria. Moe, Nils Brede. Sjoberg, Dag I.K. (mai 2020) *Daily Stand-Up Meetings: Start Breaking the Rules*. IEEE software, Vol.37 (3), s.70-77. Universitetsbiblioteket